

LAIR

B.LAIR_QT

Booru Local Archive & Indexing Repository, Qt edition

A desktop booru client. Qt6, C++, no Electron.

VERSION 0.3.0 · MPL 2.0 · re-v0

It searches several booru sites at once, downloads the originals, and keeps them in ordinary folders on your disk. Tags go into a SQLite index next to them. Delete the database and the files are still there, still sorted, still openable by anything else on the system.

What it does not do: play video, log into accounts, upload anything, or sandbox plugins.

Quickstart

Arch Linux:

```
makepkg -si
```

Gentoo — drop the ebuild into a local overlay:

```
mkdir -p /var/db/repos/local/media-gfx/blairqt
cp blairqt-0.3.0.ebuild /var/db/repos/local/media-gfx/blairqt/
ebuild /var/db/repos/local/media-gfx/blairqt/blairqt-0.3.0.ebuild manifest
emerge -av media-gfx/blairqt
```

`blairqt-9999.ebuild` builds from git master instead. `USE=python` pulls an interpreter for script plugins; `USE=test` runs the suite during the merge.

Anything else — needs Qt 6 (Widgets, Network, Concurrent, Sql, LinguistTools), CMake 3.21+ and a C++20 compiler:

```
cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Release
cmake --build build
./build/blairqt
```

On first start a welcome screen asks which sources to load. Nothing explicit is enabled unless you pick it. Type tags, hit Enter, drag the slider if the thumbnails are the wrong size. Right-click a post to download it or to blacklist a tag.

Sources

Six generic parsers — Danbooru, Gelbooru/dapi, Moebooru, e621, Philomena, Szurubooru — cover dozens of sites without a class per site. **Add custom booru** probes a URL and works out which one fits.

Several sources can be merged into one gallery. Paging, infinite scroll and autoscroll are three ways to move through results; picking one disables the controls that stop making sense.

Cloudflare-gated Gelbooru sites accept a cookie and User-Agent you paste in. Sites with no API at all can go through the bundled HTML-scrape bridge — those posts arrive without tags, rating or dimensions, which is the price of scraping HTML.

Every request, status code and response snippet lands in the network log dock. When a site misbehaves, that dock is the first place to look.

Library

Downloads are indexed in SQLite with an FTS5 tag index and written into real nested folders — the same folders Dolphin or Thunar show. Move files between them from inside the app and the tags follow; re-downloading a post you already have is detected and skipped.

Each image also gets a hidden tags sidecar next to it — `image.png` gets `.image.png.txt`, one line of space-separated tags. It follows the file when you move it, and it is what lets another tool (or you, with `grep`) read the tags without touching the database. The metadata is stored as `.txt` as well, because why not — txt is cheap.

Search the library by tag, filter by folder, with or without subfolders. Zoom opens local files instantly, animated GIFs included.

Blacklist

A tag blacklist, global or per source, editable in Settings or straight from a post's context menu. Sites that understand `-tag` get the exclusions pushed into the query; the rest are filtered before they reach the grid.

Matching is exact: `solo` does not hide `solo_focus`. It can apply to the library too, if you want the same rules on what you already downloaded.

Plugins

A new booru engine is one Python file. No C++, no rebuild:

```
def build_search_request(base_url, tags, page, per_page) -> dict:
    """Return {"url": ..., "headers": {...}}."""

def parse_response(body, base_url) -> list[dict]:
    """Return post dicts: id, file_url, preview_url, sample_url,
    post_page_url, tags (list), rating, score, width, height."""

def supports_tag_exclusion() -> bool:      # optional
    """True if the site understands "-tag", so the blacklist can be
    applied in the query instead of only filtering the results."""
```

Add it through **Add custom booru** → *Plugin script*; with Auto-detect on, the script is tried automatically. Tracebacks show up in the dialog log and the network log dock. Plugins need `python3` on `PATH`, and some need their own modules — the Newgrounds example in `examples/` wants `argon2-cffi`.

> **Plugins run with your full user permissions. Only add scripts you trust.**

HTML-scrape bridge

For Gelbooru-engine sites with no `dapi`. Not started automatically:

```
pip install flask cloudscraper beautifulsoup4
python3 tools/html_scrape_bridge.py
```

It listens on `127.0.0.1:8765`. Tick *This site has no API (dapi)* when adding the source.

Appearance and language

Themes are JSON over `QPalette`; 20 curated ones ship with the app and the system default is one click away. Interface languages: Polish, English, German, Danish, Russian, following the system language when it is one of those. Window and dock layout survive a restart.

Safety

The app writes inside its own data folder and folders you explicitly grant, and nowhere else. That is application discipline, not a sandbox — a plugin you add runs as you do. Stored API keys and cookies (`custom_sources.json`) are written owner-only (0600).

Tests

No network, no visible window:

```
ctest --test-dir build --output-on-failure
```

They cover blacklist matching, tag input, response parsing, library paging and the rule that a plugin never blocks the GUI thread.

Data locations

- Settings: `~/.config/blairqt/blairqt.conf`
- Library DB, sources, theme: `~/.local/share/blairqt/blairqt/`
- Thumbnail cache: `~/.cache/blairqt/`

License

Copyright (c) 2026 re-v0. Mozilla Public License 2.0 — see the `LICENSE` file in the source tree, which is the unmodified Mozilla text.

MPL copyleft is per file: you may combine these files with code under another license, but modifications to the files themselves stay MPL and stay open.
